Programming Language Design

Module 11: Language Interpretation

This is an open educational resource.
Feel free to submit fixes, improvements, and new material here.

Last Time

We talked about some real examples of important typeclasses.

First: [what's a typeclass]?

Second: [what typeclasses did we talk about and what are their important abilities]?

This time

It's time. Time to have the talk.

Monads...

There comes a time in every aspiring functional programmer's life in which they have to finally have to learn about monads.

Why? Because we want to actually execute our lisp code, and we can't do it without them.

But monads are a meme. People joke about no one being able to understand them. They are like pointers in C for non-C-programmers: a challenge to understand that must be overcome to master the language.

Why are pointers hard? Because they come with an algebra. You have to understand that the `&` and `*` operators are inverses of one another. This is not trivial: it's like learning a branch of mathematics that only applies to that one formal language.

Monads... (2)

This is different though. Monads aren't just a Haskell feature, they are a feature in tons of languages.

They are extremely important, and once you understand them, the pattern will appear in many many more places:

- Error handling in Rust is monadic
- Exception handling in Java is a (poorly implemented) monad.
- Async promises in Javascript (and many languages) are monads.
- Optional types in C# (and many languages) are monads.

Monads are a core design pattern in computing. You can encode pretty much any kind of computation in them. Once you understand them, you have a powerful tool.

What's a monad?

I hope you're excited to learn monads now! Monad is a term from category theory. Let's search the internet to find out what they are:

> All told, a monad in X is just a monoid in the category of endofunctors of X
>
> - Saunders Mac Lane, *Categories for the Working Mathematician*

Oh...

We know some of those words

We know what a monoid is.
We also know what a functor is (from a Haskell perspective).
There actually is a clue here that explains what a monad is.

But I think it's a mistake to get too abstract too quick.

Instead of deriving monads from endofunctors or whatever, let's motivate their existence. We're going to learn why Haskell came up with this idea.

Doing IO

Remember that Haskell is a pure functional language.

It is also a language with lazy evaluation, which forces it to be pure.

So now it's finally time to address the question: how do you actually **do** things?

- How do you draw the triangles?
- How do you access the internet?
- How do you play the sound?
- How do you roll the dice?

Is Haskell really incapable of *doing* things?

No, actually. You can make video games, servers, media players, and whatever else in Haskell. We just have to have a bit of a paradigm shift.

Don't *do* IO, *describe* it!

In a Haskell program, everything must be pure. No mutation.

However, *outside* a Haskell program, anything goes!

Haskell programs are run by a *runtime*. A runtime is basically a bunch of code that is needed to run a program in a high-level language. Pretty much every high level language has a runtime (but systems languages make it optional).

Haskell's runtime is called rts, and it is written in C.

The runtime is what connects Haskell code to the wider world. It's what actually *does* things.

Don't *do* IO (2)

So how do we do IO in Haskell?

We don't. We build a program that will do IO in the future, and we give that program to the runtime to actually execute.

So a Haskell program is really a meta-program. It's a program that builds a program.

In modern Haskell, the type of that program is `IO`. An `IO` is a program that will do input output when the runtime system executes it.

This explains a shocking fact: `main` in Haskell is not a function. Isn't that weird? In a functional language, `main` is not a function? It's a value. A value of type `IO ()`: `main :: IO ()`. It's a value representing a program that will later be executed.

What is an IO?

`IO` is a datatype, but we can't look inside it. It's what we call an *opaque* type.

Normally, we can use pattern matching to break a data type into its constructors, but the constructors for an IO are not exported from their module.

Therefore, the data inside it that stores the program is like a private variable.

Instead, we construct `IO`s using publicly visible functions. These functions are wide-ranging enough to do basically anything you could want to do, including calling C functions.

You already know one such function...

`putStrLn`

Here's the classic Haskell hello world:

```
main :: IO ()
main = putStrLn "Hello, World!"
```

Now we can finally clear up some things you may have wondered...

- As stated before, `main` is not a function. It is an `IO ()`.
- An `IO a` is a program that will do something and then return an `a`. In this case it is returning `()`, which is pronounced "unit". It's kind of like `void`.
- `putStrLn :: String -> IO ()`. That is, it's a function that takes a string, and return a program that will print that string and then not return anything.
- The reason we use `=` for `main` is that `main` *really is* that IO program.

What about more complex programs?

Okay, but presumably we don't just want to print "Hello" and then leave.

What if we want to print more than one thing?

Then you do this:

```
main = putStrLn "hello" *> putStrLn "world"
```

Remember `*>`? It was to combine applicative functors and "execute" both of them, keeping the result of the second one. In this case, they both don't return anything interesting.

So `IO` is an applicative functor, and because of that, we can take two programs and run them one after the other!

This new paradigm

The ability to "concatenate" programs is interesting. We haven't seen it before.

Suppose I have two programs:

```
printHello :: IO ()
printHello = putStrLn "hello"
printWorld :: IO ()
printWorld = putStrLn "world"
main = printHello *> printWorld
```

It's like being able to concatenate functions in c without creating a new function.

```
void printHello() { puts("hello"); }
void printWorld() { puts("world"); }
int main = printHello + printWorld + return 0; // won't work
```

But hang on...

I can hear the protest.

"Surely I can just do this!"

```
void printBoth() {
    printHello();
    printWorld();
}
```

Yes, you can. That function will do both other functions in sequence.

But it is not an expression: it is a statement block. So you have to define a new function any time you want to sequence two other functions.

You also can't change what the semicolon means. We *can* change what `*>` means. E.g., we could define a new kind of function that automatically logged every statement.

Nicer notation

It is here that we can explain the `do` block you've seen in my handouts.

The `do` block enables Haskell to pretend to be a normal language.

These two `main`s are equivalent:

```
main = putStrLn "hello" *> putStrLn "world!"
main = do
    putStrLn "hello"
    putStrLn "world"!
```

You can put the little `*>` in there at the end of each line like a little semicolon if you want to, but `do` does it for you instead.

Questions?

But what about input

We can take input, too: `getLine :: IO String`

`getLine` has type `IO String`. That is, it's a program that will eventually give a string.

But...how do we get the result *out*?

This is the first stumbling block many people face when they try to do IO in Haskell:

```
main = do
    putStrLn "Enter your name: "
    let name = getLine
    putStrLn $ "Hello, " ++ name -- type error
```

The problem is that `name` is not a string. It's a program that will eventually return a string when it runs...and we can't run it!

How do I get the string out?

Okay, if `name` is an `IO String`, how do I pull the `String` out of the `IO`?

That's the neat thing: you don't:

It actually doesn't exist.

The program hasn't been run yet!

And `Applicative` functors, while cool, can't fix this problem. The `*>` operator isn't powerful enough to do anything but run two programs one after the other.

So...uh...are we sunk? No, but now we actually *have* to learn monads.

Typeclass Monad

```
class Applicative m => Monad m where
    (>>) = (*>)
    return = pure
    (>>=) :: m a -> (a -> m b) -> m b
```

First, the small things:

- There's an operator `>>` that is identical to `*>`. `>>` uses the monad like an applicative functor, but since monads *are* applicative functors, it's the same as `*>`. Applicative functors are newer than monads, so this is a historical relic that is still used in most monad tutorials. `putStrLn "hello" >> putStrLn "world"` is just another way of writing `putStrLn "hello" *> putStrLn "world"`
- Every monad has a constructor named `return`. Again, so does every `Applicative`, and since all monads are also applicatives, you could just use `pure`. Historical.

The big thing: the "bind" operator `>>=`

The big thing that makes monads special is this operator, pronounced "bind":

```
    (>>=) :: m a -> (a -> m b) -> m b
```

Let's look at its type. It takes a program that returns an `a`, a function that takes `a`s and uses them to construct a program that returns a `b`, and then applies that function to produce a program that returns a `b`.

This allows us to stitch two programs together where one of them produces output that the other consumes. We can use it to fix our input problem from before:

A basic monadic program

```
main :: IO ()
main =
    putStrLn "Please enter your name: " >>
    getLine >>=
        (\name -> putStrLn $ "Hello, " ++ name)
```

Here, there is a `>>` after `putStrLn` to stitch it together with `getLine`. But we use `>>=` after `getLine` because we need more complicated functionality. We stitch it with a function that will eventually receive a string when `getLine` is run, and it will use that string to produce a new program, which prints `Hello` along with that string.

So we stitched a program to a program, and a program to a function which produces programs.

This program will have the desired effect. Try it!

No, it really is hard

This isn't expected to be easy. There's a lot going on and you haven't had a good chance to learn it yet.

I will make a promise, though: this is a learnable skill. It becomes effortless to work with monads.

Let's start by comparing them to functors and applicative functors...

Comparison between functor types

```
(<$>) :: Functor f =>        (a -> b)   -> f a -> f b
(<*>) :: Applicative f => f (a -> b)   -> f a -> f b
(=<<) :: Monad m =>          (a -> m b) -> m a -> m b
```

(note: `function =<< monad` is equivalent to `monad >>= function`)

- A functor has the ability to take a function to change what's inside it.
- An applicative has the ability to take a function that's already inside it and apply it to a value that's already inside it
- A monad has the ability to take a function and use it to create a new monad based on what's inside.

Monads are the most powerful of these, because the function can build a new monad from scratch, instead of modifying what's already in the monad.

"bind" is key

The `>>=` operator is key to understanding monads in CS. Let's write its type again:

```
(>>=) :: Monad m => m a -> (a -> m b) -> m b
```

"Take a monad that will produce an `a`. Pass that `a` to a function that will produce a new monad that produces a `b`. Return this new monad."

This operator is overloadable. Every monad does something different when you bind them. It's like if you could make the semicolon do something different in C depending on whatever behavior you wanted to inject into your program.

However, it's a little awkward to use...

The bind operator is awkward

Let's get a name and password together...

```
main =
    putStrLn "enter your character's name: " >>
    getLine >>= (\name ->
        putStrLn "enter this character's password: " >>
        getLine >>= (\pass ->
            putStrLn $ "registering character " ++ name ++ " with password " ++ pass
        )
    )
```

It works but...every time we want to get input, we have to create a lambda function to receive that input.

This is awkward. Luckily, `do` notation has a solution.

Using do

Here is how we write that same program using `do`:

```
main = do
    putStrLn "enter your character's name"
    name <- getLine -- this is the >>= equivalent
    putStrLn "enter your character's password"
    pass <- getLine
    putStrLn $ "registering " ++ name ++ " with password " ++ pass
```

Isn't that nice? We can pretend Haskell is a normal language!

We can even use braces and semicolons: `do { print "hi." ; print "bye!" }`

Monads (and applicatives) really are programmable semicolons!

`do` looks like some complicated syntax feature, but it follows simple rules.

Do's rules

```
do
    action1
    action2
```

is equivalent to

```
do {
    action1;
    action2;
}
```

which is equivalent to

```
do { action1 ; action 2 }
```

The last semicolon is optional

Do's rules (2)

```
do
    name <- resultReturningAction
    action -- we can use "name" now, but we don't have to
```

Is equivalent to

```
resultReturningAction >>= (\name -> action)
```

```
do -- consider this:
    a <- blip
    b <- blop
    something a b
-- same as
blip >>= (\a -> blop >>= (\b -> something a b))
```

Do's rules (3)

We can still use `let` inside of a `do` block. You can use `in` if you want, but if you don't, the name you create is visible to the rest of the `do` block, as if the whole thing was in the `in`

```
do
    let x = 7
    let y = 8
    print (x + y)
```

or

```
do
    let x in
        let y in print (x + y)
```

Do's rules (4)

But `<-` is different from let.

`let x = y` means "x is a new name for y until the end"
`x <- y` means "create a new program that will take the result from `y` and feed it to a function in which the result is named `x`. The rest of the `do` block is this function.

```
do
    x <- y
    ...
```

Means

```
y >>= (\x -> ...)
```

Do's rules (5)

This is why `<-` is necessary when getting user input.

If we do this:

```
do { let name = getLine ; putStrLn $ "hello, " ++ name }
```

Then `name` is not a string. It's a program that will return a string. And we cannot concatenate programs to strings.

Do's rules (6)

Instead, if we do this:

```
do { name <- getLine ; putStrLn $ "hello, " ++ name }
```

Then the part after the `;` is a function body:
`(\name -> putStrLn $ "hello, " ++ name)`, and in this function, *name* is a `String`.

Let me repeat that: when you use `>>=` or `<-`, the type of the variable will be the type *inside* the monad, because the value inside is the one being fed to the function.

```
getLine >>= (\name -> ... )
```

In the above function, `name` is a string, because `getLine` is an `IO String`.

Finally, more intuition

These are some more intuitive notes that may or may not trigger an "aha".

`>>` or `*>` stitches two programs together to create a new program.

`a >> b` is a new program that will do the same thing as `a`, then discard the result, then will do the same thing as `b`.

`>>=` stitches a program to a function that will generate a new program. The program and the function become a new program.

`>>=` allows later actions to depend on the results of previous ones.

Monad knowledge check 1

So let's see some concrete examples of how to apply this.

Knowledge check 1: ask the user to enter a number. Then, print out whether or not the number is prime.

Most of the code we need *does not* need to be monadic. But the input/output part does.

[How do we go about this?]

Monad KC 1 answer

Don't immediately jump into monads. Good old functions are still very useful.

We can use functions to determine if a number is prime:

```
divides :: Int -> Int -> Bool
divides a b = a `mod` b == 0

-- integer square root
isqrt :: Int -> Int
isqrt i = floor $ sqrt $ int2Double i

isPrime :: Int -> Bool
isPrime i = i > 1 &&
    not (any (divides i) [2 .. (isqrt i)])
```

An integer is prime if it is bigger than 1 and if none of the numbers from `2` up to its square root can divide it. That logic is handled by `isPrime`.

Monad KC 1 answer (2)

Okay, but how do we interact with the user?

```
main :: IO ()
main = do
    putStrLn "please enter a number: "
    line <- getLine
    let num = read line :: Int
    putStrLn (line ++
        if isPrime num
            then " is prime."
            else " is not prime.")
```

You can think of a `do` block as a "monad builder". Here, we first ask for a number, then we `bind` a program that will get a line and feed the results forward. We convert it to an integer, and then we print it and whether it's prime. [Why `<-` versus `let`?]

Monad KC question 2

I was going to use this one on a quiz, but it's so important I decided to make it part of the lecture material so that every year of students could see it.

There's a classic programming question that is commonly used as a warmup question in technical interviews.

It's called *FizzBuzz*. It's also a verb: *FizzBuzzing* someone means testing them on a basic task.

So, let's see that basic task...

Monad KC question 2 (2)

For every integer from 1 to 100 (inclusive), print "Fizz" if it is divisible by 3, "Buzz" if it is divisible by 5, "FizzBuzz" if it is divisible by both, and if none of the above apply, just print the number. Correct output sample:

```
1
2
Fizz
4
Buzz
...
13
14
FizzBuzz
...
```

[Ideas?]

Monad KC question 2 answer

Some thoughts:

- Whether a number is printed as itself, or one of the strings, is dependent entirely on that number and no other state.
- Therefore, we can write a function that converts a number to the correct string.
- And then, we can map that function to every element of a list of integers.
- Finally, we need to somehow print those

Let's work on the easy part first. [Shall we solve it together?]

Monad KC question 2 answer (2)

```haskell
fizzbuzzify :: Int -> String
fizzbuzzify i
    | i `mod` 15 == 0 = "FizzBuzz"
    | i `mod` 3 == 0 = "Fizz"
    | i `mod` 5 == 0 = "Buzz"
    | otherwise = show i
```

Here, we use a little number theory trick to test whether the number is divisible by 3 and 5. If it is divisible by both, it will be divisible by 15.

You don't have to use this trick, of course. `i `mod` 3 == 0 && i `mod` 5 == 0` works.

If we get past the checks, we just use `show` to convert the number to a string of itself. Remember that `show` is Haskell's equivalent of `.toString()`.

Monad KC question 2 answer (3)

Then we can map all the numbers like this:

```
fizzbuzzify <$> [1 .. 100]
```

This creates a big list of strings, `["1", "2", "Fizz", etc.]`

But how do we print all of them?

[thoughts?]

One way to do it

Suppose we did this:

```
putStrLn <$> ["1", "2", "Fizz", ...]
```

Remember, `putStrLn` is a function, so we can map it. The result is a list of actions:

```
[putStrLn "1", putStrLn "2", putStrLn "Fizz", ...]
```

But we don't want 100 independent programs that print stuff. We want one program that we can return from main.

How can we collapse those 100 programs into 1?

Two options

First, we can use one of the folds. It doesn't matter which one, since `>>` is associative.

```
main = do
    let strings = fizzbuzzify <$> [1 .. 100]
    let prints = putStrLn <$> strings
    foldl (>>) (return ()) prints
```

Wait, what's that `return ()`?

`return` is the universal constructor for monads. It creates an empty monad that does nothing except "return" the given value. So `return 7` can be an `IO Int` that, when run, will `return` 7 and do nothing else.

`foldl` requires a starting value, so we use `return ()` to create an empty program to `>>` with the first `putStrLn`.

A nicer option

It turns out that `IO` is a monoid, so we have a nicer option.

Instead of using `foldl`, which makes us provide a default value, we can use `mconcat`.

```
mconcat prints
```

That's it. That will squash the whole list of prints into a single program that we can return.

More familiar option

If those are too weird, we can even use a for loop!

Wait? Really? Haskell has for-loops?

Not as syntax elements. But monads are so flexible, there is a `for` function that does everything a for loop does. In most languages, this would need to be built-in, but Haskell has it as a function.

It's called `forM_`. The `M` is for monad, and the `_` means "discard the output of the last value". There is also `forM` if we want to keep it:

```
forM_ strings (\string ->
    putStrLn string
) -- or just forM_ strings putStrLn
```

Look how much it looks like a for loop!

More familiar option

There's a more "Haskelly" version, `mapM_` which does the same thing as `forM_` but the argument order is reversed:

```
main = do
    let strings = fizzbuzzify <$> [1 .. 100]
    mapM_ putStrLn strings
```

It's taken a long time, but hopefully the pieces are starting to fall into place as to how you can do normal programming language things in Haskell.

However, behind the scenes, even statements are algebraic objects that you can manipulate, giving you enormous flexibility in extending the language.

Almost no other language gives you this much flexibility...
Almost...except one...

Not so fast

But before we talk about Lisp, let's do some `IO` monad drills.

Remember, `IO` is just one monad of many, but it's the main one we need to use to make Haskell do anything outside the program, so it's kind of important.

Make sure none of these programs are too hard. There will be a quiz!

Monad Problems

1. Calculate and print the first 100 fibonacci numbers.
2. Ask the user to enter their name, then print how many characters are in it.
3. Ask the user to enter their name, then convert it to Spongebob case (so Grant becomes gRaNt)
4. Ask the user to enter a first name and a last name. Paste them together and print the result. [unworked]
5. Ask the user to enter the two leg lengths of a right triangle. Print the length of the hypotenuse. [unworked]
6. Ask the user to enter some digits, and print whether it could maybe be a PIN (4 digits), a Zip code (5 digits), or a phone number (7 or 10 digits). [unworked]

Monad fibos

```
fibos :: [Int]
fibos = 0 : 1 : zipWith (+) fibos (tail fibos)

main :: IO ()
main = forM_ (take 100 fibos) print
```

The `fibos` list uses lazy evaluation. The remainder of the list is created by zipping the list itself with its own tail. This is more efficient than applying the definition.

We use `forM_` to loop over the first 100 fibonacci numbers and call `print` on them.

Monad name chars

```
main :: IO ()
main = do
    putStrLn "please enter your name: "
    name <- getLine
    putStrLn $ "your name has " ++ show (length name) ++ " characters."
```

This one is a bit easier. We just read the name in and construct the message.

We can take advantage of the fact that monads are a kind of functor though:

```
main :: IO ()
main = do
    putStrLn "please enter your name: "
    nameLen <- show . length <$> getLine
    putStrLn $ "your name has " ++ nameLen ++ " characters."
```

Monad name chars (2)

Applying fmap (`<$>`) to a monad creates a new monad in which its return value has been transformed by the given function.

`getLine` is an `IO String`
`length <$> getLine` is an `IO Int` that returns the length instead of the line.
`show <$> length <$> getLine` will convert the length `Int` into a `String`.
`show . length <$> getLine` is equivalent because of functor laws.

Spongebob case

One last one for us to do together. Spongebob case.

To do this one, I zip the string with a list of ints that each correspond to the index of each character in the string.

If it is an even index, we want to make it lowercase. If odd, uppercase.

```
spongebobCase :: Int -> Char -> Char
spongebobCase index char
    | even index = toLower char
    | otherwise = toUpper char
```

Spongebob case (2)

Now we just map that function to the string and provide all the indices

```
toSpongebob :: String -> String
toSpongebob = zipWith spongebobCase [0..]

main :: IO ()
main = do
    putStrLn "please enter your name: "
    name <- toSpongebob <$> getLine
    putStrLn $ "nIcE To mEeT YoU " ++ name
```

Its fine if we provide an infinite list of ints. `zip` and `zipWith` stop as soon as the end of the shorter list is reached, so if the name is finite, the `zipWith` result will be finite.

Questions?

What about lisp?

Lisp actually has this much flexibility.

It has a feature that is analogous to monads, but works completely differently.

In the same way that general grammars are turing complete, but they work in a completely different way, Lisp *macros* can fulfill the same requirements as Haskell *monads*. In fact, they are perhaps a little more powerful.

But we have a long way to go. Let's learn how to comple and run our lisp programs.

What do we need?

Remember that Slisp, the language we're going to make for this class, is Simple Lisp. A Lisp program broadly looks like this:

```
(print "hi")
(print (+ 2 2))
(print "bye")
```

Those parentheses denote lists:

- `()` is the empty list (not shown)
- ("hi") is the list containing "hi"
- (print "hi") is the list containing a symbol `print` and a string `"hi"`

(an actual Common Lisp programm often uses `format` command to print. It's kind of like the lisp equivalent of `printf`, if you're looking at lisp programs online.)

The program is a list

Lisp is unique in that it is the first *homoiconic* programming language.

Homoiconicity is the property that the code of the program is, itself, available in a data structure inside the language. That is, **code is data**.

The basic data structure of Lisp is the list. And the lisp program itself is a lisp list.

Specifically, it is a list of lists. Each inner list is a command, where the first symbol is a function or macro (I will explain this term in a second) to be called, and the remaining list elements are the arguments.

Why is this helpful?

It lets you have *real* macros

"Oh macros, I know what those are from C. Like `#define` and stuff"

Yes, those are macros. *Text* macros. The worst kind.

They work by copying and pasting text together. This is very simple and powerful, but also very unweildy. You can't just say "here is a list of code lines: print the code first, then execute it", because that would require parsing the lines, and the macro happens *before* parsing.

But what if the code wasn't just a string, what if you could access the parse tree?

In lisp, the program *is* the parse tree. So you can modify parts of the program as if they were any old list.

Macros let you add new features

The main benefit of macros is that they let you add new features to the language.

A macro in Lisp is a function that runs *at compile time*, that takes and returns a list.

That means, we can take raw, unevaluated code and modify it to add new features.

Lisp dialects like Common lisp are famous for basically having every feature. Want object oriented programming? They have classes. Want currying? They can do that. Want monads? They can have those too.

You can use macros to basically add new features to the language after it came out.

We will talk about macros later. For now, I just want to motivate why we're writing an interpreter for this weird language. It isn't just a raw learning exercise: we are learning a powerful language that can do pretty much everything, even ergonomically.

Back to slisp

Let's consider a very basic program in our dialect:

```
(print 2)
(print (+ 2 2))
```

We intend for this to first print 2, then print 4 (because `(+ 2 2)` means `2 + 2`)

But how do we *run* that code?

Well first, we need to parse it.

We already wrote a parser for project 3. What would it return on that code?

The result of parsing

Something like this:

```
VList [ VList [ VSym "print", VInt 2 ],
          VList [ VSym "print", VList [VSym "+", VInt 2, VInt 2 ]]]
```

In fact, let's adopt lisp-style lists. Lists in this syntax are called *S-expressions*:

```
((print 2) (print (+ 2 2)))
```

Notice: that's literally just the program with an extra pair of parentheses around it.

The program itself is just a list. And we want to *execute* that list.

Let's make that look nicer

Now that we're using s-expressions, let's write a function that turns our `Values` into the corresponding s-expression.

We could make a custom instance of the `Show` typeclass, but I prefer to have the default `Show` available for debugging.

Instead, here's a function to "pretty print" a value...

Pretty printing

```
prettyPrint :: Value -> String
prettyPrint (VList list) = "(" ++ unwords (prettyPrint <$> list) ++ ")"
prettyPrint (VInt i) = show i
prettyPrint (VSym s) = s
```

The base cases are when a value is an int or a symbol. We "show" the int to convert it into a string, or just return the symbol.

When the value is a list, we recursively pretty print all its values, and use "unwords" to paste the values together with a space in between.

Lastly, we cap it off with a '(' and ')' on either end.

Now what?

Okay, we have parsed a list, and we can print it out to inspect it. How do we run it?

We can't. Remember that Haskell is a functionally pure language. Running the code would mean printing stuff out and changing variables and all sorts of unsafe nonsense.

But we can compile a program to run later.

That is, we can take the program we parsed and compile it into an `IO`. Then, return that `IO` back to the runtime to execute it.

Compilation?

Wait, compiling? I thought we were making an interpreter?

The distinction is less clear than you might think.

A *pure* interpreter, like an old school, Commodore BASIC-style interpreter, would parse and execute each line separately.

If a line contained a `GOTO` to send you to another line, it would then re-parse the tokens and execute that line.

This is obviously not very fast, modern languages that we consider "interpreted" don't actually do that.

Compilation? (2)

Python, for example, does have a compiler.

You write some python, it tokenizes, then parses it, but then it compiles what it parsed into an intermediate language.

The difference between an "interpreted" language and a compiled language isn't that the modern interpreted language never compiles. It's that its compiler does not compile all the way down to machine language, and it typically runs immediately after compilation.

That's what we're going to build. Our "compiler" will build an `IO` that, when executed, will run the program that the user submitted.

Compiling into an `IO`

This is our compile function's type:

```
eval :: Value -> IO Value
```

We're calling it "eval" because it produces an `IO` that will evaluate to a value.

Evaluating a number, like `7`, just produces `7`. But evaluating a list runs it.

We want the lisp function:

```
(+ 2 2)
```

To compile into a program that, when it is run, produces `VInt 4`.

Compiling into an `IO` (2)

For integers and symbols, it is very simple:

```
eval (VInt i) = return (VInt i)
eval (VSym s) = return (VSym s)
```

Remember, `return` is a function that produces a monad that doesn't actually do anything except return the given value.

So `return (VInt i)` produces an `IO Value` when it is executed.

The correct way to "evaluate" an int or a symbol is to just return it.

But what about lists? We have to *execute* those. So we have to produce a function that will run the function in the list, and then

What does it mean to execute it?

Now we come to the key point: the lists that we have encountered in other classes don't do anything. They are data structures. They sit around, holding data.

So we need some kind of strategy that tells us how to interpret that data as code to be executed.

The lisp execution strategy is simple:

- The program is a list of lists
- For each sub-list, if it starts with a symbol...
    - call the corresponding function with the remaining elements of the list as args.
    - otherwise crash.

Let's start with `+`

Okay, how do we run `(+ 2 2)`?

Any time we execute a list, we pull the first element to determine what to do:

```
commandArgs :: [Value] -> (String, [Value])
commandArgs (VSym name : args) = (name, args)
commandArgs c =
    error ("invalid command list: command" ++
        prettyPrint (VList c) ++ "does not start with a symbol.")
```

This function takes a list like `(+ 2 2)` and breaks it into a command `+`, and a list of arguments. The result of running `commandArgs` on `(+ 2 2)` is:

```
("+", [VInt 2, VInt 2])
```

Let's start with `+` (2)

Then, we check the function. Is it a built-in function? If so, we make an `IO` that will execute it. If it's not built-in (maybe it's a function the user made), we look it up. We'll handle that case in project 5.

So, let's check for built-in functions:

```
evalBuiltin :: String -> [Value] -> IO Value
evalBuiltin "+" args = ...
evalBuiltin "print" args = ...
evalBuiltin command _ = error $ "unrecognized command: " ++ command
```

This function takes a string that tells us the name of the built-in we want, and then it outputs a program that will execute that function on the given arguments.

What should "+" do?

Let's start with `+` (3)

```
evalBuiltin "+" args = compileAdd $ expectInt <$> args
```

We have a helper function called `evalAdd`, but it expects a list of integers. Lisp is dynamically typed, so the values we pass to it could be anything. We don't want to accept lists or symbols, we just want to add ints. So we use `expectInt` to force it.

```
expectInt :: Value -> Integer
expectInt (VInt i) = i
expectInt v = error $ "expected int but got " ++ prettyPrint v

evalAdd :: [Integer] -> IO Value
evalAdd args = return $ VInt $ sum args
```

`evalAdd` takes a list of arguments, computes their sum, and returns a program that will produce that sum.

What about `print`?

At this point, we can add numbers. If you run `evalFunction` on `"+"` and `[2, 2]`, you can get an `IO Value` that will return `4`. So programs like this can be compiled

```
(+ 2 2)
```

But that program doesn't *do* anything. We need some way to print the results.

If all we needed was to compute values, we wouldn't really need a compiler that returned `IO Value`. We could just evaluate a list directly, with a function like this `eval :: Value -> Value`. That's how the first version of this course worked when we used the *Forth* programming language.

However, I want to be able to print now, so it's time to get more familiar with monads!

Understanding print

This is why our compile function returns an `IO Value`. `IO a` means "program that will do a bunch of things and then finally return an `a`.

So an `IO Value` is a program that will potentially print or read input and then return a lisp value when it's done.

So, our print function can return an `IO Value` which prints, and then returns whatever it wants (we don't really care about the return value here).

Understanding print (2)

Let's extend our `evalBuiltin` function:

```
evalBuiltin "print" args = evalPrint args
```

And then define `evalPrint`:

```
evalPrint :: [Value] -> IO Value
evalPrint args = do
    let pretty = prettyPrint <$> args -- pretty is a string list
    let str = unwords pretty -- unwords joins a list together with spaces
    putStrLn str -- actually do the printing
    -- if there is one argument to print, then that is its value
    -- otherwise its value is a list of all its args
    return $ if length args == 1 then head args else VList args
```

[Let's spend a moment understanding it...]

Finally, a compile function

Now that we have our two built-in functions, let's have a function that compiles values.

```
eval :: Value -> IO Value
eval (VList list) = do
    let (command, args) = commandArgs list
    compiledArgs <- mapM eval args
    compileBuiltin command compiledArgs
eval other = return other
```

If we pass it a list, it will break it into its command and its arguments.

Then, it wlil run `eval` on all its arguments. `mapM` stands for "map monad". Here, it takes itself, and runs it on every value. `mapM` then takes the list of `[IO Value]` and stitches them together into an `IO [value]`. That is, it takes a list of actions, and returns one action that returns each of the produced values.

`mapM`

What does `mapM` really do? Imagine we had a list like this: `["1", "hi", "bye"]`

Suppose we wanted to print all those things.

We could do this:

```
map print ["1", "hi", "bye"] == [print "1", print "hi", print "bye"]
```

But that's just a list of actions. How can we produce one action that does all of them?

```
foldl (>>) (return ()) [print "1", print "hi", print "bye"] == -- this works
return () >> print "1" >> print "hi" >> print "bye"
```

The end result is a single program that prints 3 things. (The initial `return ()` is a program that does nothing. We just need it to be there as a starting value for `foldl`)

`mapM` (2)

Folding a bunch of monads into one monad is so common, there are functions for it:

```
sequence :: Monad m => [m a] -> m [a] -- for when we want the intermediate values
sequence_ :: Monad m => [m a] -> m () -- for when we don't
```

`sequence_` is just `foldl (>>) (return ())`
That is, `sequence_ [print 1, print 2] == print 1 >> print 2`

The version without the underscore returns a list of intermediate values at the end. The results of running each program in the list of programs. Since `print` does not produce a value, this is not helpful. But if we have programs that *do* produce values (like `eval`), it's helpful. Here's an example of what it does:

```
sequence [return 1, return 2, return 3] == IO that returns [1, 2, 3]
```

`mapM` (3)

So what if we compile a list of arguments? Now we have a list of programs:

```
eval <$> [VInt 2, VInt 2] ==
    [eval (VInt 2), eval (VInt 2)] ==
    [eval (VInt 2), eval (VInt 2)]
```

So we can sequence them together so that we now have a single program that produces all that data:

```
sequence . eval <$> [VInt 2, VInt 2] == return [2, 2]
```

And that's what `mapM` does. It just maps a monadic function to a collection, and then sequences it into a single action that returns the collection:

```
mapM f c == sequence . f <$> c
```

`mapM` for compiling the args

So, let's return to our function that gives evaluators:

```
eval :: Value -> IO Value
eval (VList list) = do
    let (command, args) = commandArgs list
    evaledArgs <- mapM eval args
    evalBuiltin command evaledArgs
eval other = return other
```

Here, `mapM eval args` just says "build me an `IO` that will first run `eval` on all the arguments, and then return the results of doing that.`

We use `<-` to use those results in the program we're building. We then compile the built-in function with the fully evaluated args.

Questions?

Especially about `mapM`?

It seems to work

At this point, we can patch up our main `IO` to compile and run our lisp code:

```
main :: IO ()
main = do
    program <- getContents
    let tokens = tokenize program
    let parsed = parseProgram tokens
    let compiled = mapM_ eval parsed
    compiled -- actually evaluate by returning the compiled evaluator
```

Here, we use `mapM_` which, instead of returning an `IO [Value]`, returns an `IO ()`.

We don't actually want those values anymore. The type of `main` is `IO()`, so we have to throw them away.

However, all of the actions (like printing) will be executed, which is what we want.

It seems to work (2)

And in fact, we can pass this code:

```
(print 2)
(print (+ 2 2))
(print (+ 2 2 2))
(print (+ (+ 2 2) (+ 2 2)))
```

And I get this result:

```
2
4
6
8
```

Which is correct.

A program is more than a sequence of operations

But there's a basic thing that real programs need to do that we can't do yet: branch.

Branching is key to computer science. If our program cannot branch, it can't be Turing complete, and Turing completeness is kind of a bare-minimum bar to shoot for here.

So, let's add `if` expressions to the language.

Important point: Lisp does not distinguish between expressions and statements. A statement is just an expression that has a side effect. Even `print` returns something.

Conditions

First, every `if` has a condition, right?

Like, in C, `if (x == 0) { ... }`

How can we add `==` testing to our language?

Deriving `Eq`

The easiest way is to derive `Eq` for our values:

```
data Value =
        VSym String
    |   VInt Integer
    |   VList [Value]
        deriving (Show, Eq)
```

Now this works: `(VInt 7 == VInt 7) == True` and `(VInt 7 == VInt 8) == False`

We can even allow inequalities with `deriving (Show, Eq, Ord)`

This will make it so that when we use inequalities, they apply to the values inside the constructors: `VInt 7 <= VInt 8 == True`.

Total ordering

This is a total ordering, which means that *all* values participate in the relation.

That is, symbols can also be compared to integers.

When we use `deriving Ord`, Haskell will make it so that all instances of the first constructor are less than the second, all instances of the second are less than the third, and so on.

So all symbols are considered less than all integers: `VSym x <= VInt y == True` always.

Lists are also always greater than symbols and ints, but what about other lists?

Lists are compared lexically, in dictionary order. So list a is less than list b if its first element is strictly less, or if its first element is equal and its second element is strictly less, etc.

Adding `==`

So now, we want to add a new command, `==` (and then soon its friends)

First, let's have the ability to require exactly 2 arguments:

```
expectPair :: [Value] -> (Value, Value)
expectPair [arg1, arg2] = (arg1, arg2)
expectPair _ =
    error "wrong number of arguments. expected 2."
```

And now we can add it:

```
evalEq :: [Value] -> IO Value
evalEq args =
    let (a, b) = expectPair args
    in return $ if a == b then VInt 1 else VInt 0--uses 1 and 0 for true and false
```

Bools

For the purpose of this simple language, I'm going to consider 0 and the empty list to be false, and anything else to be true, but 1 by default.

This is not how classic lisp works. Lisp clasically used the symbol `t` for true, and "nil" (which was the empty list) for false.

We could even add a boolean kind of value if we wanted, to distinguish it from integers and symbols.

[Are there any language design implications to one choice or another?]

Adding more relations

For your project, you'll need to add `<=`, `/=` etc.,

To do this, I like to define an intermediate function that I can compile any binary relation between `Value`s with:

```
compileRelation :: (Value -> Value -> Bool) -> (Value, Value) -> IO Value
compileRelation rel (a, b) = return $
    if a `rel` b then VInt 1 else VInt 0
```

This injects any function between two values (like `==`) and converts its result to `1` for true and `0` for false

But using it requires a small refactoring. Can you see how to use it?

Questions?

Turing completeness

The requirements for a language to be turing complete are:

1. Sequence (running a list of statements in order)
2. Branching (conditional jumps or ifs)
3. Repetition (loops, backwards jumps, general recursion)

We have a primitive form of sequencing: the program is a list of commands.

Let's work on branching. How do conditionals work in lisp?

Introducing `cond`

Classic lisp uses this kind of conditional

```
(cond
    ((== x 0) (print 0))
    ((== x 1) (print 17))
    ((print 1111)))
```

`cond` (short for "condition") is equivalent to an if-else.

In the code above, it first tests whether `x` is `0`, and if so, the result is `(print 0)`
Then it tests if `x` is `1`, and if so, the result is `(print 17)`.

If all the conditions fail, it evaluates the last expression. It doesn't have a condition.

You will need to implement `cond`, but in this lecture, we'll implement a simpler version, called `when`.

Introducing `when`

When is like a simple if-statement with no else-branch.

```
(when (== (+ 2 2) 4) (print 1))
```

Here, we will only print 1 if the condition is true. If it's false, we don't do anything.

We can also have multiple

At this point, I like to return the value inside the condition regardless so we can use when as a conditional trace:
`(print (when 0 (print 1)))` ends up return zero from the `when`, which gets printed.

Can we make this be a built in function? Let's try and see what happens.

How could we make `when`?

Can we make this be a built-in function?

Like, suppose we added a `cond` function the same way we added `==` and `print`. Would that work?

Let's just try it and see what happens. Here's an `evalWhen`:

```
evalWhen :: Value -> [Value] -> IO Value
evalWhen (VInt 0) tail = return $ VInt 0
evalWhen (VList []) tail = return $ VList []
evalWhen _trueCondition tail = evalBlock tail
```

What about `evalBlock`?

`evalBlock`

This program will run every command in a block:

```
evalBlock :: [Value] -> IO Value
evalBlock commands = do
    evald <- mapM eval commands
    -- if empty, return the empty list, otherwise the last value
    if null evald
        then return $ VList []
        else return $ last evald
```

It uses `mapM` to evaluate all the commands in the block, then run them all in sequence.

If there are no given commands, it just returns the empty list.

If there are commands, it returns the value of the last one executed.

Last steps?

Now we can add our `when` function:

```
evalBuiltin "when" [] = error "when without condition"
evalBuiltin "when" (condition : prog) = evalWhen condition prog
```

`evalWhen` will check the given condition. If it evaluated as true, it will run the given program. Otherwise, it shouldn't run it (`evalWhen` should just return the false condition in that case)

Does it work?

Sort of...

This code works:

```
(when (== (+ 2 2) 4) (print 4))
```

This does, in fact, print 4.

The problem is, this code also prints 5, even though it shouldn't.

```
(when (== (+ 2 2) 5) (print 5))
```

Any idea why?

Functions won't work here

The problem is, `when` cannot be a function!

Consider this C program:

```
int fake_if(int condition, int trueval, int falseval) {
    return condition ? trueval : falseval;
}
```

If we try to run it where the true and false values have side effects, it won't work:

```
int x = 0
fake_if(2 + 2 == 4, x = x + 1, x = x - 1);
```

Here, `x = x + 1` and `x = x - 1` always run, no matter whether the condition was true or false. Because *functions always fully evaluate their arguments*.

We need something else

What we need here is not a function: it's a macro.

Macros are like functions, but they see their arguments *before* they get evaluated.

In C, using `#define` for our `fake_if` example would work. But how can we do this in our language?

Adding macros

We need to update our `eval` function for commands.

Previously, it would fully evaluate all of the arguments by running itself recursively on every argument to the command.

We need this behavior to only happen for functions. Let's define a function that tells us whether something is a macro or not:

```
isBuiltInMacro :: String -> Bool
isBuiltInMacro "cond" = True
isBuiltInMacro "when" = True
isBuiltInMacro _ = False
```

Now, we can update eval to, when it's being called on a macro command, *not* evaluate all its arguments.

Evaluating macros

```
-- compile a value into an IO that returns it
eval :: Value -> IO Value
eval (VList list) = do
    let (command, args) = commandArgs list
    if isBuiltInMacro command
        then evalBuiltinMacro command args
        else do
            args' <- mapM eval args
            evalBuiltin command args'
eval other = return other
```

If the command is a macro, we simply run `evalBuiltinMacro` directly, without evaluating the arguments.

If it's not a macro, we still evaluate the args.

Evaluating `when`

```
evalBuiltinMacro "when" [] = error "when without condition"
evalBuiltinMacro "when" (condition : prog) = do
    condition' <- eval condition
    evalWhen condition' prog

evalWhen :: Value -> [Value] -> IO Value
evalWhen (VInt 0) tail = return $ VInt 0
evalWhen (VList []) tail = return $ VList []
evalWhen _trueCondition tail = evalBlock tail
```

Now, we *only* evaluate the first argument, the condition. We use the `<-` to get the result of running the condition. Then we use the result to determine if we should evaluate the tail or not.

With this change, it works.

Questions?

The quiz (not what you think)

There is enough here to keep you busy. You'll need to implement `cond` among other things for project 4.

I know that monads are hard, so I want to give you time to study. Therefore, I will give an easier quiz that anyone who has been paying attention will do well on.

It's going to quiz you on hypothetical programming languages and ask you to evaluate and classify them and their features.

This will be very easy for anyone who has been paying attention.

Quiz format

The quiz will be a handful of fairly easy multiple-choice, short answer, true false, or light coding questions.

That's it.

Sample quiz 1

Each question is worth 20%

1. Which of these paradigms best describes Haskell?
   a. Imperative
   b. Functional
   c. Object-oriented
   d. Query-based
2. What is the basic unit of an imperative program (i.e., a line like `x = 10;`)
3. What is the basic unit of a functional program (i.e., a string like `x + y`)
4. What is another major difference between imperative vs. functional programming?
5. List a feature that, if a language had it, would make it more imperative.

Sample quiz 1 answers

1. Functional
2. The statement
3. The expression
4. Functional languages prefer avoiding mutation, and sometimes make it impossible.
5. Mutating assignment, while loops, print statements that execute right away, lots of choices.

Sample quiz 2

Each question is worth 20%

1. List an imperative programming language
2. List a feature that this language has that Haskell does not have
3. Why do you think Haskell doesn't have this feature?
4. List a feature that Haskell has that the other language doesn't have.
5. Why do you think the other language does not have this feature?

Sample quiz 2 answers

1. C++
2. Classes
3. Haskell doesn't need to control mutation, because it's already impossible.
4. Monads
5. Implementing it would require higher-kinded types, and managing the memory usage of combining a lot of values with `>>` would probably make the interface a decent bit more complex.

You know the drill

Ask an AI to generate more of these. You can feed it the markdown files of the other lectures to give it more ammunition.

Questions?